

Ontology and SkillGraph Documentation

AIDF Project. PI Liu.

1 Requirements

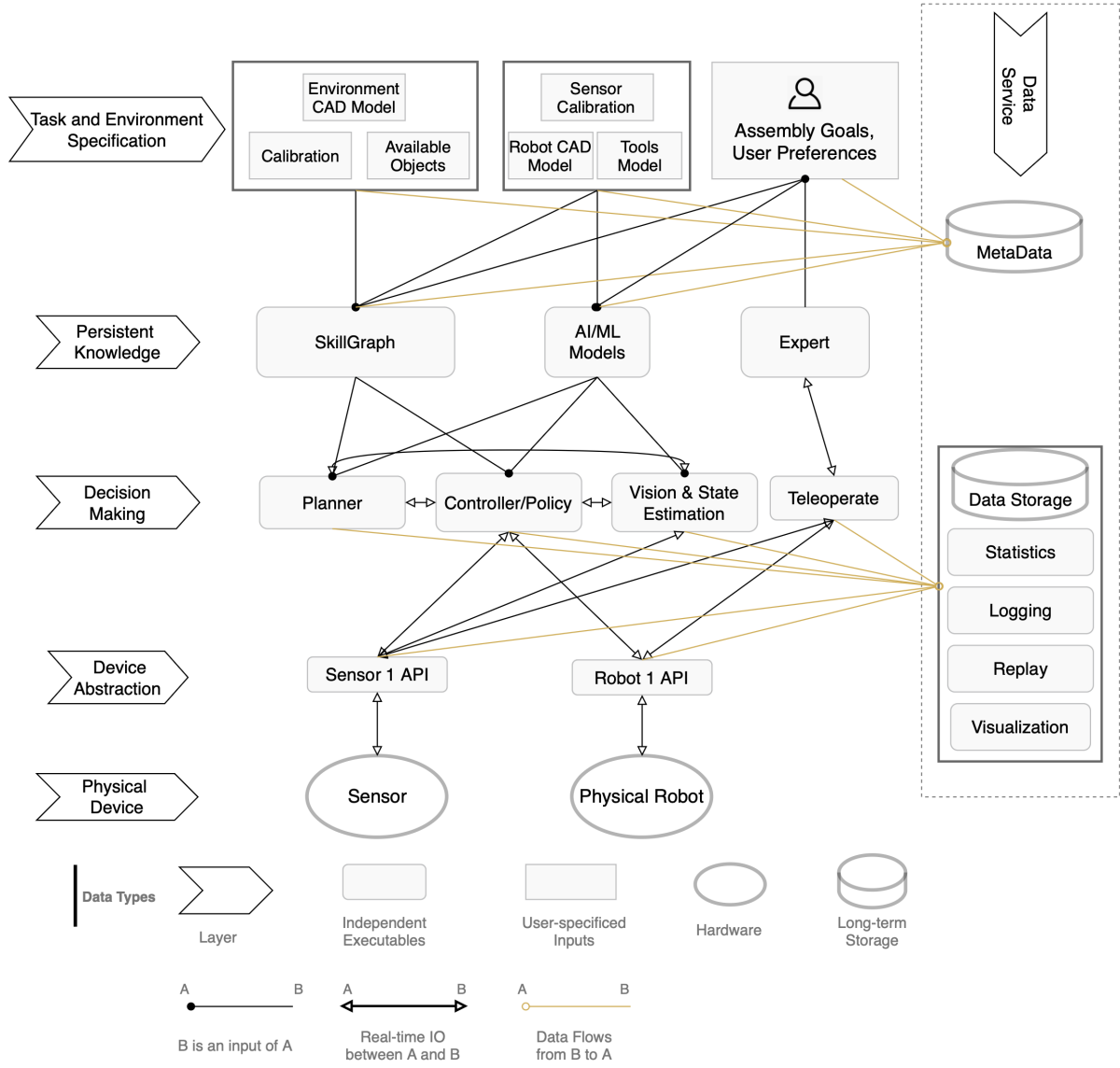


Figure 1: Sequential high-level ontology of the robot skill graph.

In modern manufacturing, robotic assembly cells have become increasingly crucial as manufacturing systems grow more complex and autonomous. An ontology provides a standardized semantic foundation that enables seamless integration of diverse components and workflows within the assembly ecosystem. A good ontology should have the following requirements:

1. (Structure) The ontology establishes a structured representation of knowledge and workflow, and a systematic approach to organize robotic algorithms. The structure should also enable the incorporation of new concepts, relationships, and capabilities as technology evolves. A good structure and clear relationship also make it accessible to non-experts, allowing them to understand and interact

with complex robotic systems without deep technical expertise.

2. (Interface) The ontology should provide a common language and interface to facilitate efficient communication between software components and robotic systems. It should also define a common language for user requirement specification.
3. (SkillGraph) The ontological standardization should allow for the creation of reusable skillgraphs, where domain-specific capabilities and algorithms can be mapped and composed in a modular fashion.
4. (Data) Furthermore, the ontology serves as a comprehensive data management framework, enabling structured generation and organization of both operational data and associated metadata, thereby enhancing the reusability and transferability of manufacturing knowledge across different assembly scenarios and applications.

2 High-level Ontology

Figure 1 shows a high-level layout of our proposed ontology for robotic assembly. On a high-level, the ontology is organized sequentially into five layers and a shared layer. These layers are Task Specification, Persistent Knowledge, Decision Making, Device Abstraction, Physical Device, and the shared Data Service Layer. We break them down below.

2.1 Task and Environment Specification

Task and Environment specification layer defines the setup and requirements of the robotic assembly system. This can be broadly split into two categories - environment description and task specifications. Environment descriptions include multiple user-defined components, such as the description of environments, robots, objects, hardware models, calibrations, and all known environment states. The user can also define any task specification for the overall assembly cell in the form of an assembly target or sequence, as well as any operation preferences, optimization objectives, and hyperparameters.

2.2 Persistent Knowledge

The Persistent Knowledge layer contains all necessary knowledge, skills, models, and domain expertise to support robot decision making. Hardware and task specifications from previous layers are passed as inputs to the skill graphs and any machine learning models. A special case is the Expert, which can be an experienced human or privileged algorithm that may control the decision making process in the next step. Other additional node that can be part of this layer can be a physics simulator or optimization solvers.

2.3 Decision Making

The Decision Making layer is where all robot decision-making algorithms are located and implemented in code. This layer is responsible for commanding the robot to accomplish the task. It can utilize any persistent knowledge to plan a long-horizon action plan, control the robot's behavior in real time, and estimate the state of robot and the assembly environment. The decision making layer can also be a teleoperator. Other example node that may fit this layer could be an end-to-end visual motor policy, or a collaborative human-robot decision maker.

2.4 Device Abstraction

The Device Abstraction layer is a standardized interface that bridges the physical device and any decision-making node. The software interface provides a standard way to communicate sensor information and robot commands to any decision-making node, which may be developed by different teams with different programming practices. The abstraction layer also ensures that the addition of new robots and new sensors can be scalable and should insulate the decision making from any complexity in managing multiple hardware devices.

2.5 Physical Device

The Physical Device layer represents all sensors and robots used in the assembly cell.

2.6 Data Service

The Data Service layer is a vertical layer designed to record and store execution data. It provides an easy-to-use utility service to log, replay, visualize, and compute the statistics of robot execution data. There are two categories of data - metadata and execution data. Metadata stores all task specifications, hardware setup, model hyperparameters, and the description of any skills or models used during the assembly process. On the other hand, execution data refer to any data generated during the decision making and execution processes. It should contain all data passing through the device abstraction layer, which can be visualized in a digital twin, saved, and replayed in future. Outputs and intermediary statistics from decision-making nodes, such as inference time, prediction confidence, can also be recorded.

2.7 Plans for Implementation

Currently, the layout of the high-level ontology remains a conceptual design. The goal is to implement key parts of the ontology to form a functional skeleton. Specifically, a skill graph, planner, and controller will be implemented. Some pre-existing individual parts, such as the robot API and data services, will be integrated. The framework will be tested first tested on Lego Assembly to ensure its correctness, then extended to other assembly tasks. On the other hand, we do not plan to implement the Expert-Teleoperate pipeline in this year's project. Still, this shows the high-level ontology is designed with the possibility of adding new components.

For task specifications and environment description, our plan is to extend our current JSON data structure tailored to Lego assembly to more a more general format. Currently for Lego assembly, the user specifies a set of Json files, including 1) the detailed assembly sequence, 2) robot calibration and robot kinematic and, 3) existing objects in the environment and their locations. In addition to these, a more general data structure for task specification should also store the available robot tools, sensors, object type, location of CAD models, and metadata (e.g. skill hyperparameters). Before finalizing a precise definition, we will conduct more studies on potential use cases, such as the NIST box assembly testbed, and check existing ontology in the robotics community

3 Skill Graph

The next sections in this documents provides a more detailed description of the skill graph within the overall proposed ontology. The skill graph serves as a centralized codebase for organizing the necessary robot- and environment-related assumptions and knowledge to enable intelligent integration and execution. The skill graph has five sections:

1. Physical Entities: *Robot*, *Object*, and *Environment*
2. Concepts: *Skill*, including *AtomicSkill* and *MetaSkill*.
3. Executables: *SkillExecutor* and *Algorithm*. The *Algorithm* includes *PlanningAlgorithm*, *ControlAlgorithm*, *PerceptionAlgorithm*, and *Constraint Satisfaction Generator*.
4. Evaluation Metric: *Metric*, including *Constraint Evaluator*, and *Condition Evaluator*.
5. Data Structures: *TaskParam*, *RobotState*, and *Env State*.

Besides, we also illustrate the input to the SkillGraph, symbolized as *Task*. The skill graph is demonstrated in fig. 2.

3.1 Data Types and Relationships in Skill Graph

There are several data types in fig. 2:

1. BaseClass: *Task*, *Robot*, *Object*, *Environment*, *Skill*, *SkillExecutor*, *Algorithm*, *Metric*, *TaskParam*, *RobotState*, and *Env State*

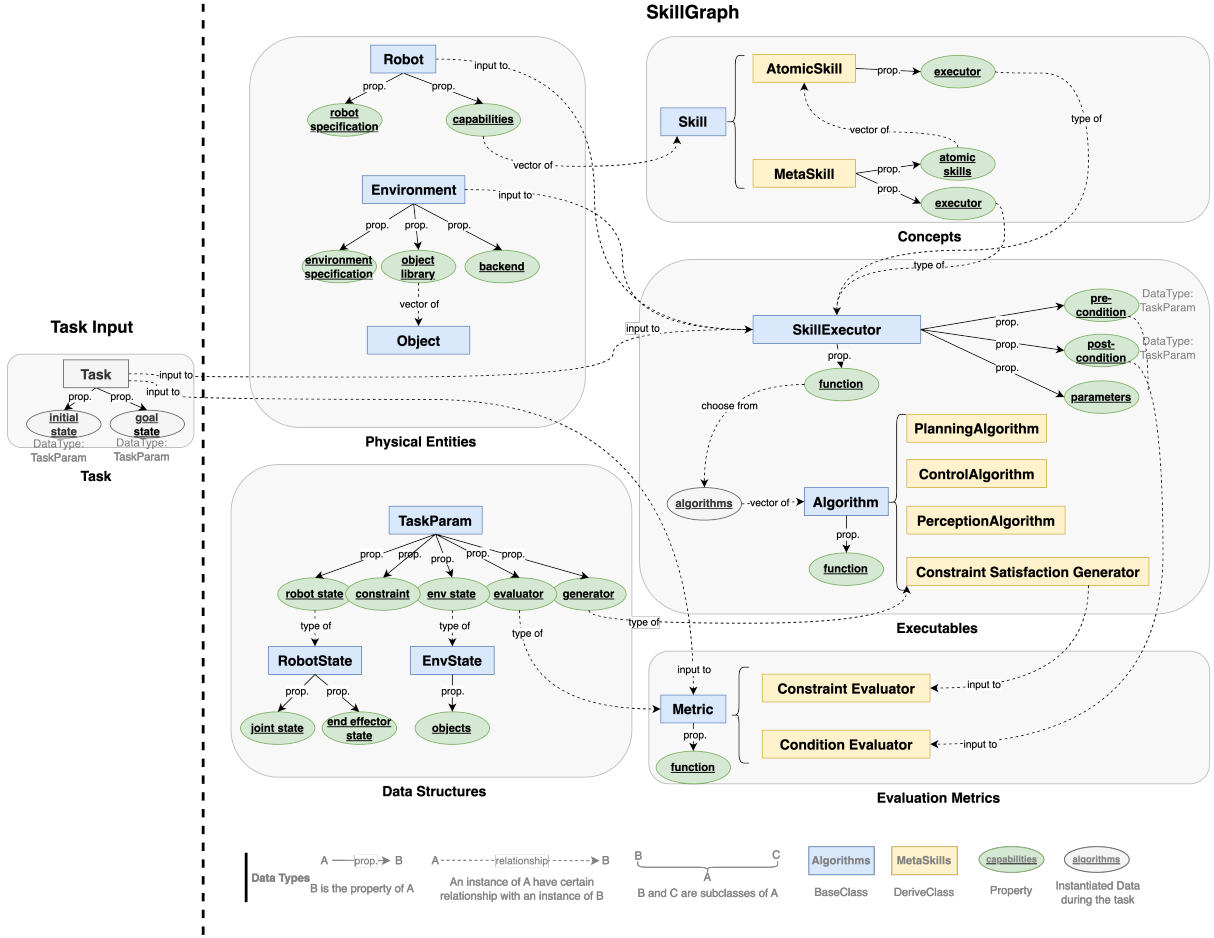


Figure 2: Symbolic Representation of Skill Graph

2. **DerivedClass:** *AtomicSkill* and *MetaSkill* are derived from baseclass *Skill*. *PlanningAlgorithm*, *ControlAlgorithm*, *PerceptionAlgorithm*, and *Constraint Satisfaction Generator* are derived from baseclass *Algorithm*. *Constraint Evaluator* and *Condition Evaluator* are derived from baseclass *Metric*.

3. **Property:** Each class might own certain properties, specified by a solid arrow with label “prop.” on it. For example, in fig. 2, *atomic skills* and *executor* are properties of derivedclass *MetaSkill*.

Each data may have certain relationship with other data, and the relationships are specified by arrows. If the arrow points from A to B, indicating that A has a certain relationship with B. There are different types of arrows:

1. A $\xrightarrow{\text{relationship}}$ B indicates that A has certain relationship with B. For example, *object library* is a vector of *Object*, and *Environment* is an input to *SkillExecutor*.
2. A $\xrightarrow{\text{prop.}}$ B indicates that B is the property of A.

The relationships are represented as labels on the arrows, and the relationships are defined as follows:

1. A *prop.* B: A has a property B.
2. A *input to* B: A is the input to B.
3. A *vector of* B: A is a vector of B.
4. A *type of* B: A’s data type is B.

4 Main Components in SkillGraph

The key components in the SkillGraph are:

- **Skill:** Comprising derived classes *AtomicSkill* and *MetaSkill*, which define the basic and composite robot skills.
- **SkillExecutor:** Defined by its four properties: *function*, *pre-condition*, *post-condition*, and *parameters*.
- **Algorithm:** Comprising derived classes *PlanningAlgorithm*, *ControlAlgorithm*, *PerceptionAlgorithm*, and *Constraint Satisfaction Genetor*.
- **Robot:** Defined by its two properties: *capabilities* and *robot specification*.
- **Object:** we allow user to define their own object type.
- **Environment:** Defined by its three properties: *environment specification*, *object library* and *back-end*.
- **Metric:** Comprising derived class *Constraint Evaluator* and *Condition Evaluator*.

To further enhance the structure and organization of the SkillGraph, we define several essential data structures:

- **TaskParam:** Represents the parameters required for task execution, can be specified by user.
- **RobotState:** Captures the current state of the robot, including its joint state and end effector state.
- **EnvState:** Capture the environment state, including all objects' locations, and status.

Currently, our code is implemented in this GitHub repository. We have defined all the header files of our C++ class according to the semantics and relationships we specify below. Class implementations of all .cpp files and specific skills for Legos, are still in progress.

In the following sections, we would give a detailed definition of each components and their relationships with other components.

4.1 Skill

We define any robot action or robot decision that satisfies the following requirements as a skill: 1) can be executed in a real robot or a sensor, 2) has semantic meaning, and 3) can be planned on. We store those skills in base class *Skill*, and divide them into *AtomicSkill* and *MetaSkill*. Each meta skill is composed by a set of atomic skills, arranged in a sequence or in some partial order.

4.1.1 AtomicSkill

AtomicSkill is a derived class from base class *Skill*, and it has one property:

1. executor: type of *Skill Executor*.

The entity of *AtomicSkill* is shown in fig. 3. Currently, we define five instances of *AtomicSkill*: Pick, Place, Support, Transit, and Align. Each atomic skill has its own *executor*, whose datatype is *SkillExecutor*.

AtomicSkills					
ID	Pick	Place	Support	Transit	Align
<u>executor</u>	Algorithms:: PerformPick	Algorithms:: PerformPlace	Algorithms:: PerformSupport	Algorithms:: PerformTransit	Algorithms:: PerformAlign

Figure 3: Entity of AtomicSkill

4.1.2 MetaSkill

MetaSkill is another derived class from base class *Skill*, and has 2 properties:

1. executor: type of *Skill Executor*.
2. atomic skills: a vector of type *AtomicSkill* that constitutes current meta skill.

The entity of *MetaSkill* is shown in fig. 4. Each instance of *MetaSkill* is comprised of atomic skills, and each meta skill also has its own executor. Currently we have five meta skills: PlaceWithSupport and Handover, PickAndPlace, PickAndPlaceWithSup, PickHandoverAndPlace to group common action sequences and reduce complexity of downstream planning.

MetaSkills					
ID	PlaceWithSupport	Handover	PickAndPlace	PickAndPlaceWithSup	PickHandoverAndPlace
executor	Algorithms::PerformPlaceWithSupport	Algorithms::PerformHandover	Algorithms::PerformPickPlace	Algorithms::PerformPickPlaceWithSup	Algorithms::PerformPickPlace
atomic Skills	[AtomicSkills::Place, AtomicSkills::Support]	[AtomicSkills::Place, AtomicSkills::Support]	[AtomicSkills::Pick, AtomicSkills::Place]	[AtomicSkills::Pick, AtomicSkills::Place, AtomicSkills::Support]	[AtomicSkills::Pick, AtomicSkills::Handover, AtomicSkills::Place, AtomicSkills::Support]

Figure 4: Entity of MetaSkill

4.2 Skill Executor

Skill Executor is defined as a class that wraps the logic, effect, and requirements of skills execution in the above section. It has 4 properties:

1. function: the underlying execution function of each skill would include instances from *PlanningAlgorithm*, *ControlAlgorithm*, *PerceptionAlgorithm*, and *Constraint Satisfaction Algorithm*.
2. pre-condition: a *TaskParam* that measures what requirements the current state must satisfy before executing the skill.
3. post-condition: a *TaskParam* that measure what the state would become after the execution of certain skill.
4. parameters: the parameters of the skill execution function or the pre/post-conditions.

The entity of *Skill Executor* is shown in fig. 5. As mentioned in section 4.1, each skill corresponds to one skill executor stored in class *Skill Executor*. Every skill executor has three inputs: robot, environment, and task.

SkillExecutor							
ID	Perform Pick	Perform PickAndPlace	Perform Support	Perform Transit	Perform Align	Perform PlaceWSupport	Perform Handover
function	Perform Pick	Perform PickAndPlace	Perform Support	Perform Transit	Perform Align	Perform PlaceWSupport	Perform Handover
pre-condition	PreCondition Pick	PreCondition PickAndPlace	PreCondition Support	PreCondition Transit	PreCondition Align	PreCondition PlaceWSupport	PreCondition Handover
post-condition	PostCondition Pick	PostCondition PickAndPlace	PostCondition Support	PostCondition Transit	PostCondition Align	PostCondition PlaceWSupport	PostCondition Handover
parameters	PickParams	PickAndPlace Params	Support Params	Transit Params	Align Params	PlaceWSupport Params	Handover Params

Figure 5: Skill Executor

4.3 Algorithm

The class *Algorithm* is a class that implements common robotics planning/control/perception algorithms. Base Class *Algorithm* have 4 derive classes: *PlanningAlgorithm*, *ControlAlgorithm*, *PerceptionAlgorithm*, and *Constraint Satisfaction Generator*. As mentioned in section 4.2, each skill executor function needs to choose its own planning, control, perception, and constraint satisfaction algorithms. There may be multiple algorithms suitable for each skill type. Those algorithms are stored in its corresponding derived classes. And every algorithm has its own function, as shown in fig. 6.

PlanningAlgorithms			ControlAlgorithms		
ID	RRT	LegoPlan	ID	PID	MPC
<u>function</u>	RRT	LegoPlan	<u>function</u>	PIDtracking	MPCtracking

(a) Planning Algorithm

PerceptionAlgorithms			Constraint Satisfaction Generator		
ID	Object Detection	Precise Align	ID	Feasible Set Generator	Grasp Pose Sampler
<u>function</u>	Object Detection	Precise Align	<u>function</u>	FindAllFeasible Object	Find one Grasp Pose

(c) Perception Algorithm

(d) Constraint Satisfaction Generator

Figure 6: Algorithm

4.3.1 PlanningAlgorithm

PlanningAlgorithm is in charge of planning robot trajectories, and currently we have two instances of planning algorithms: RRT and LegoPlan, as shown in fig. 6a.

4.3.2 ControlAlgorithm

ControlAlgorithm is in charge of tracking the planned robot trajectories, and currently we have two instances of control algorithms: PID and MPC, as shown in fig. 6b.

4.3.3 PerceptionAlgorithm

PerceptionAlgorithm is in charge of perceiving information from the environment, and currently we have two instances of perception algorithms: Object Detection and Precise Align, as shown in fig. 6c.

4.3.4 Constraint Satisfaction Generator

Constraint Satisfaction Generator is in charge of generating specific data in order to satisfying certain constraints specified by the user, like feasible set and grasp pose shown in fig. 6d.

4.4 Robot

Class *Robot* is the abstract entity that describes the physical robot and its characteristics. The class has two properties:

1. robot specification: a json file that includes the detailed specification of each robot.
2. capabilities: a list of instances of *Skill* that the robot can perform.

Robot		
ID	GP4	Fanuc
<u>robot specification</u>	GP4.json	Fanuc.json
<u>capabilities</u>	1. Skills::Pick 2. Skills::Place 3. Skills::Support 4. Skills::Press 5. Skills::Transit	1. Skills::Pick 2. Skills::Place 3. Skills::Support 4. Skills::Press 5. Skills::Transit

Figure 7: Robot

The entity of *Robot* is shown in fig. 7. Currently, we define two instances of *Robot*: GP4, and Fanuc. Both robots have the same capabilities of Pick, Place, Support, and Transit.

4.5 Object

Currently, we allow users to flexibly define their own *Object* class without imposing explicit restrictions on its properties, as the required object attributes may vary across different tasks.

4.6 Environment

Class *Environment* is the abstract entity that describe the physical assembly cell and its characteristics. The class has three properties:

1. environment specification: a json file that specifies the details of the environment.
2. object library: a vector of type *Object* that specifies all objects that can be interacted with in the environment
3. backend: a middleware or simulator that handles the control of the environment.

Environment		
ID	EnvLego	EnvNist
<u>environment specification</u>	EnvLego.json	EnvNist.json
<u>object library</u>	vector <Object::objects>	vector <Object::objects>
<u>backend</u>	Moveit	Mujoco

Figure 8: Entity of Environment

The entity of Environment is shown in fig. 8. Currently, we define two instances of *environment*: EnvLego for Lego manipulation, and EnvNist for NIST test case assembly.

4.7 Metric

Class *Metric* defines any quantitative measure used to verify any task, object, or robot requirements that may constrain skill execution. Currently, our base class *Metric* has 2 derived classes: *Constraint Evaluator*, and *Condition Evaluator*. The entities of these two classes are shown in fig. 9.

Constraint Evaluator			
ID	GeneralEval	LegoEval	GraspPoseEval
<u>function</u>	isConstraintSatisfied	isBlockTypeCorrect	isGraspPoseFeasible

(a) Entity of Constraint Evaluator

Condition Evaluator		
ID	PreConEval	PostConEval
<u>function</u>	precondition_ evaluator()	postcondition_ evaluator()

(b) Entity of Condition Evaluator

Figure 9: Metric

4.7.1 Constraint Evaluator

Constraint Evaluator is in charge of evaluating whether an instance of *Constraint Satisfaction Generator* satisfies the constraints of the given task defined by its *TaskParam*.

4.7.2 Condition Evaluator

Condition Evaluator is in charge of evaluating whether the precondition or postcondition of a given *SkillExecutor* is satisfied.

4.8 TaskParam

Data Structure *TaskParam* has several properties:

1. robot state: Represents the current state of the robot, in type of *RobotState*.
2. constraint: Defines task-specific robot-state constraints.
3. env state: Represents the current state of the environment, in type of *EnvState*.
4. evaluator: Checks if certain task-specific constraints or conditions are satisfied, in type of *Metric*.
5. generator: Generates robot state according to given constraint, in type of *Constraint Satisfaction Generator*.

4.9 RobotState

Data structure *RobotState* has several properties:

1. joint state: Specifies robot joints state.
2. end effector state: Specifies end-effector state.

4.10 EnvState

Currently, our data structure *EnvState* mainly focus on one property:

1. objects: Specifies the state of objects, including their locations, directions, and other status.

5 Implementation Details

In this section, we present the structure of our code and explain how each component from the previous sections is implemented within it.

5.1 Code Structure and Component Implementation

```
/AIDF
|— config/
|— planner/
|— skillgraph/
|   |— api/ (class and data structure definitions)
|   |   |— Utils/
|   |   |— skillgraph.hpp (implement the structure of the skill graph)
|   |   |— algorithms.hpp (implement SkillExecutor and Algorithm)
|   |   |— backend.hpp (implement the backend definitions in Environment)
|   |   |— environment.hpp (implement Environment)
|   |   |— metrics.hpp (implement Metric)
|   |   |— objects.hpp (implement Object)
|   |   |— robots.hpp (implement Robot)
|   |   |— skills.hpp (implement Skill)
|   |   |— tasks.hpp (implement TaskParam)
|   |   |— data_structure.hpp (implement Data Structures)
|— src/
```

- | | |— corresponding cpp files...
- | |— include/
- | | |— lego/ (lego-specific implementations)
- | | |— Instance Implementation.hpp files